

CCSAR

Instructions Encoding

Arquiteturas de Computadores para Sistemas Críticos
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Representing Instructions

- **Assembly language is convenient for humans to read**
 - However, digital circuits understand only 1's and 0's
- **We are now ready to see the difference between:**
 - The way humans instruct computers and...
 - ...the way computers see instructions
- **Instructions are encoded in binary**
 - Therefore, a program written in assembly is translated from mnemonics to a representation using only 1's and 0's
 - Called **machine code**

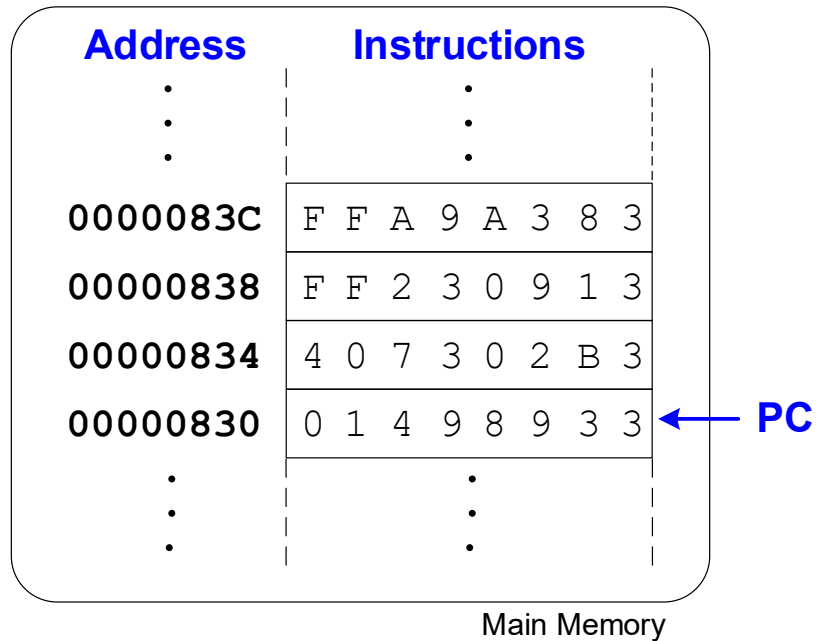
Representing Instructions

Assembly Code

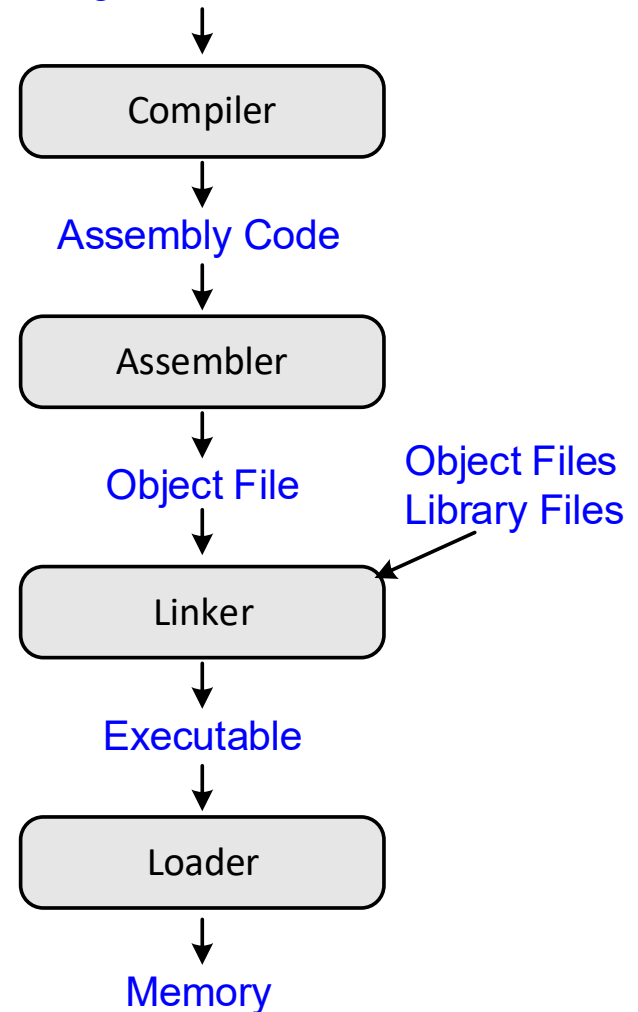
```
add  s2, s3, s4
sub  t0, t1, t2
addi s2, t1, -14
lw   t2, -6(s3)
```

Machine Code

```
0x01498933
0x407302B3
0xFF230913
0xFFA9A383
```



High Level Code



Representing Instructions

- In the base RV32I ISA, instructions are encoded as 32-bit instruction words
 - Even though some instructions may not require all 32 bits of encoding, **variable-length instructions would add complexity**
- **So, for every instruction, we need to:**
 1. Fetch one 32-bit word
 2. After fetching an instruction, the default next PC is $PC + 4$
- **Regularity!**

Representing Instructions

- **Each segment of an instruction is called a `field`**
 - Each piece of an instruction can be considered as an individual number, and placing these numbers side by side forms the instruction
- **Instruction fields have names to make them easier to discuss:**

■ <code>op</code>	operation code (<code>opcode</code>)
■ <code>rd</code>	destination register number
■ <code>rs1</code>	first source register number
■ <code>rs2</code>	second source register number
■ <code>funct3</code>	function variant (extends <code>opcode</code>)
■ <code>funct7</code>	function variant (extends <code>opcode</code>)
■ <code>imm</code>	immediate value

Representing Instructions

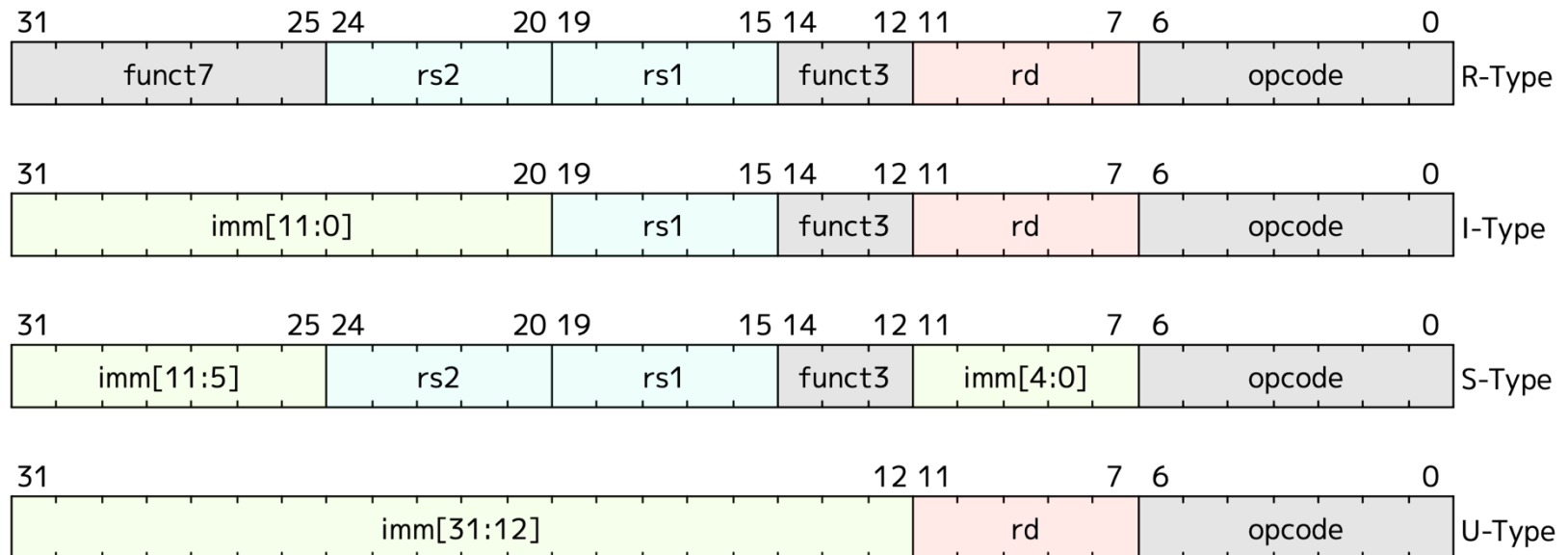
- Fields directly encode the register numbers without additional naming conventions
 - The 32 registers of RISC-V are just referred to by their number, from 0 to 31

Assembly

	Field Values						Machine Code						
	funct7	rs2	rs1	funct3	rd	op	funct7	rs2	rs1	funct3	rd	op	
add s2, s3, s4	0	20	19	0	18	51	0000,000	10100	10011	000	10010	011,0011	(0x01498933)
add x18,x19,x20													
sub t0, t1, t2	32	7	6	0	5	51	0100,000	00111	00110	000	00101	011,0011	(0x407302B3)
sub x5, x6, x7													
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	

Representing Instructions

- In the base RV32I ISA, there are **four core data-processing instruction formats (R/I/S/U)**, used for arithmetic, memory access and immediate values
 - Regularity among instructions and, thus, simpler decoder hardware, while also accommodating different instruction needs



Tools for Exploring RISC-V Instructions

■ RISC-V Instruction Encoder / Decoder

- <https://luplab.gitlab.io/rvcodecs/>

■ Key features

- Encode: Assembly → Binary / Hexadecimal
- Decode: Binary / Hexadecimal → Assembly
- Supports the main RISC-V instruction formats and extensions
- Visually maps assembly operands to their corresponding instruction fields
- Supports both register names ($x5$) and ABI names ($t0$)
- Useful for verifying manual encoding and decoding

RISC-V Instruction Decoding

- **Formats are distinguished by the values in the opcode field**
 - Each format is assigned a distinct set of `opcode` values so that the hardware knows how to treat the rest of the instruction

Instruction	Format	funct7	rs2	rs1	funct3	rd	opcode
add (add)	R	0000000	reg	reg	000	reg	0110011
sub (sub)	R	0100000	reg	reg	000	reg	0110011

Instruction	Format	immediate	rs1	funct3	rd	opcode
addi (add immediate)	I	constant	reg	000	reg	0010011
lw (load word)	I	address	reg	010	reg	0000011

Instruction	Format	immediate	rs2	rs1	funct3	immediate	opcode
sw (store word)	S	address	reg	reg	010	address	0100011

R-format Instructions

- **Revisiting the add instruction**
 - Adds the values in registers `rs1` and `rs2`, stores result in `rd`

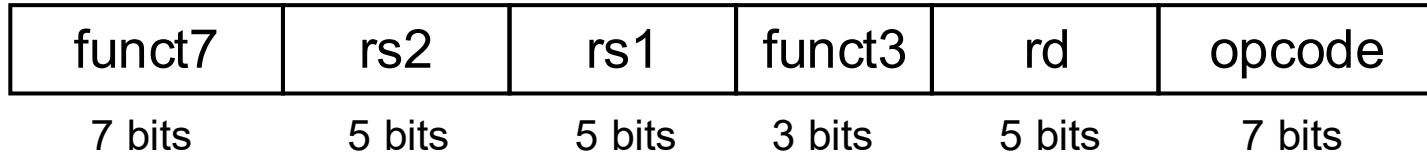
C Code

```
a = b + c;
```

RISC-V assembly code

```
# s0 = a, s1 = b, s2 = c  
add s0, s1, s2
```

R-format Instructions



- **Operate directly on registers**

- rs1, rs2: source registers
- rd: destination register
- opcode: the operation code
- funct7, funct3 (7 and 3-bits, respectively)
 - With opcode, tells CPU what operation to perform
 - Can distinguish up to 1024 function-code combinations within a single R-format opcode

R-format Instructions

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

add x9, x20, x21

0	21	20	0	9	51
0000000	10101	10100	000	01001	0110011

0000 0001 0101 1010 0000 0100 1011 0011₂
(0x015A04B3)

R-format Instructions

- Notice that the registers are in the opposite order in the assembly and machine language instructions
 - E.g., the assembly instruction `add s2, s3, s4` has `rd = s2` (18), `rs1 = s3` (19), and `rs2 = s4` (20)
 - These registers are listed left to right in the assembly instruction but right to left in the machine instruction

Assembly

Field Values

Machine Code

	funct7	rs2	rs1	funct3	rd	op	funct7	rs2	rs1	funct3	rd	op	
<code>add s2, s3, s4</code>	0	20	19	0	18	51	0000,000	10100	10011	000	10010	011,0011	(0x01498933)
<code>add x18,x19,x20</code>													
<code>sub t0, t1, t2</code>	32	7	6	0	5	51	0100,000	00111	00110	000	00101	011,0011	(0x407302B3)
<code>sub x5, x6, x7</code>													
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	

Assembly

Field Values

Machine Code

	funct7	rs2	rs1	funct3	rd	op	funct7	rs2	rs1	funct3	rd	op	
<code>sll s7, t0, s1</code>	0	9	5	1	23	51	0000 000	01001	00101	001	10111	011 0011	(0x00929BB3)
<code>sll x23,x5, x9</code>													
<code>xor s8, s9, s10</code>	0	26	25	4	24	51	0000 000	11010	11001	100	11000	011 0011	(0x01ACCC33)
<code>xor x24,x25,x26</code>													
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	

I-format Instructions

■ Revisiting the `addi` and `lw` instructions

- `addi` adds an immediate value to a register ($rd = rs1 + imm$)
- `lw` loads a 32-bit word from memory into a register, with an immediate value as the offset ($rd = Mem[rs1 + imm]$)

C Code

```
// int is a 32-bit signed word
int a = -372;
int b = a + 6;

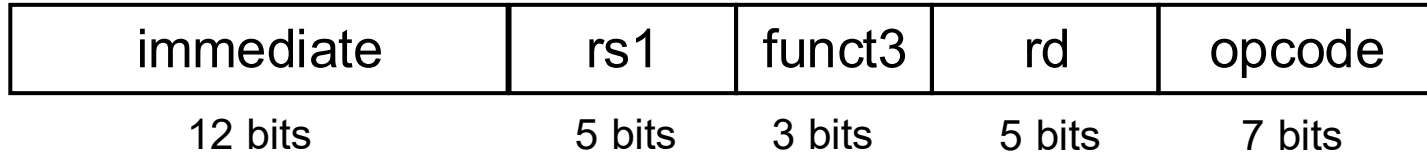
a = vec[5];
```

RISC-V assembly code

```
# s0 = a, s1 = b
addi s0, zero, -372
addi s1, s0, 6

# s0 = a, s2 = &vec[0];
lw s0, 20(s2)
```

I-format Instructions



■ Immediate arithmetic and load instructions

- `rs1`: source or base address register number
- `immediate`: constant operand, or offset added to base address
 - 2s-complement, **sign extended**
- `opcode`: the operation code
 - Simplicity favors regularity: all instructions have `opcode`
- `funct3`: the function (3-bit function code)
 - With `opcode`, tells CPU what operation to perform

I-format Instructions

- For most I-type instructions, the immediate is a 12-bit signed value (bits [31:20])
 - For immediate shifts $\text{imm}[4:0]$ is the 5-bit unsigned shift amount
 - $\text{imm}[11:5] = 0$ for `slli/srli`, while `srai` sets $\text{imm}[10]$ (instruction bit 30) = 1

Assembly

Field Values

Machine Code

```
addi s0, s1, 12
addi x8, x9, 12
addi s2, t1, -14
addi x18, x6, -14
lw t2, -6(s3)
lw x7, -6(x19)
lb s4, 0x1F(s4)
lb x20, 0x1F(x20)
slli s2, s7, 5
slli x18, x23, 5
srai t1, t2, 29
srai x6, x7, 29
```

$\text{imm}_{11:0}$	rs1	funct3	rd	op
12	9	0	8	19
-14	6	0	18	19
-6	19	2	7	3
0x1F	20	0	20	3
5	23	1	18	19
(upper 7 bits = 32) 29	7	5	6	19
12 bits	5 bits	3 bits	5 bits	7 bits

$\text{imm}_{11:0}$	rs1	funct3	rd	op	
0000 0000 1100	01001	000	01000	001 0011	(0x00C48413)
1111 1111 0010	00110	000	10010	001 0011	(0xFF230913)
1111 1111 1010	10011	010	00111	000 0011	(0xFFA9A383)
0000 0001 1111	10100	000	10100	000 0011	(0x01FA0A03)
0000 0000 0101	10111	001	10010	001 0011	(0x005B9913)
0100 0001 1101	00111	101	00110	001 0011	(0x41D3D313)
12 bits	5 bits	3 bits	5 bits	7 bits	

S-format Instructions

■ Revisiting the `sw` instruction

- Stores a 32-bit word from a register to memory ($\text{Mem}[\text{rs1} + \text{imm}] = \text{rs2}$)

C Code

```
// int is a 32-bit signed word  
vec[12] = a + vec[8];
```

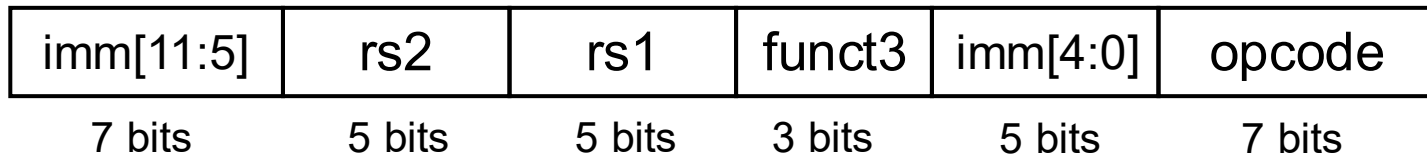
RISC-V assembly code

```
# s0 = a, s1 = &vec[0]  
lw  s2, 32(s1)  
add s2, s0, s2  
sw  s2, 48(s1)
```

S-format Instructions

- **Stores need to read:**
 - `rs1` for base memory address
 - `rs2` for data to be stored
 - The immediate offset
- **Stores don't write a value to the register file**
 - As such, there is no `rd` field
- **RISC-V design decision is to move the low 5 bits of immediate to where `rd` field was in other instructions**
 - Keep `rs1/rs2` fields in same place
 - Similarly, the `opcode` and `funct3` fields are the same size in all locations, and they are always in the same place

S-format Instructions



■ Store instructions specify:

- `rs1`: base address register number
- `rs2`: source operand register number
- `immediate`: offset added to base address
 - Split so that `rs1` and `rs2` fields always in the same place
- `opcode`: the operation code
- `funct3`: specifies the operation variant (`sb`, `sh`, `sw`)

S-format Instructions

- E.g., in `sw x7, -6(x19)`
 - Register `x19` is the base address (`rs1`)
 - `x7` is the second source (`rs2`) (the value to be stored to memory)
 - `-6` is the offset (recall that negative immediate values are represented using 12-bit two's complement notation)

Assembly

Field Values

Machine Code

```
sw t2, -6(s3)
sw x7, -6(x19)
sh s4, 23(t0)
sh x20, 23(x5)
sb t5, 0x2D(zero)
sb x30, 0x2D(x0)
```

imm _{11:5}	rs2	rs1	funct3	imm _{4:0}	op
1111 111	7	19	2	11010	35
0000 000	20	5	1	10111	35
0000 001	30	0	0	01101	35
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

imm _{11:5}	rs2	rs1	funct3	imm _{4:0}	op	
1111 111	00111	10011	010	11010	010 0011	(0xFE79AD23)
0000 000	10100	00101	001	10111	010 0011	(0x01429BA3)
0000 001	11110	00000	000	01101	010 0011	(0x03E006A3)
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	

From Assembly to Machine Code

lw	x9, 120(x10)	# x9 = A[30]	(I-format)
add	x9, x21, x9	# x9 = h + A[30]	(R-format)
addi	x9, x9, 1	# x9 = h + A[30] + 1	(I-format)
sw	x9, 120(x10)	# A[30] = h + A[30] + 1	(S-format)

120		10	2	9	3
0	9	21	0	9	51
1		9	0	9	19
3	9	10	2	24	35

From Assembly to Machine Code

lw	x9, 120(x10)	# x9 = A[30]	(I-format)
add	x9, x21, x9	# x9 = h + A[30]	(R-format)
addi	x9, x9, 1	# x9 = h + A[30] + 1	(I-format)
sw	x9, 120(x10)	# A[30] = h + A[30] + 1	(S-format)

000011110000		01010	010	01001	0000011
0000000	01001	10101	000	01001	0110011
0000000000001		01001	000	01001	0010011
0000011	01001	01010	010	11000	0100011

- Keeping the instruction formats as similar as possible **reduces hardware complexity**

32-Bit Immediate Operands

- Although constants are frequently short and fit into the 12-bit fields, sometimes they are bigger
- Use **load upper immediate** (`lui`) specifically to load a 20-bit constant into bits 12 through 31 of a register
 - Allowing a subsequent instruction to specify the lower 12 bits of the constant

```
lui rd, imm
```

- Copies 20-bit constant to left 20 bits of `rd`
- Clears right 12 bits of `rd` to 0

32-Bit Immediate Operands

- What is the RISC-V assembly code to load the value 3998976 into register x19?

- $3998976_{(10)} = 0000\ 0000\ 0011\ 1101\ 0000\ 0101\ 0000\ 0000_{(2)}$
- $976_{(10)} = 0000\ 0000\ 0011\ 1101\ 0000_{(2)}$
- $1280_{(10)} = 0000\ 0101\ 0000\ 0000_{(2)}$

lui x19, 976

0000 0000 0011 1101 0000	0000 0000 0000
--------------------------	----------------

addi x19, x19, 1280

0000 0000 0011 1101 0000	0101 0000 0000
--------------------------	----------------

- As discussed in the last lecture, the pseudo-instruction **li** (load immediate) does this
 - The assembler breaks large constants into pieces and then reassemble them into a register

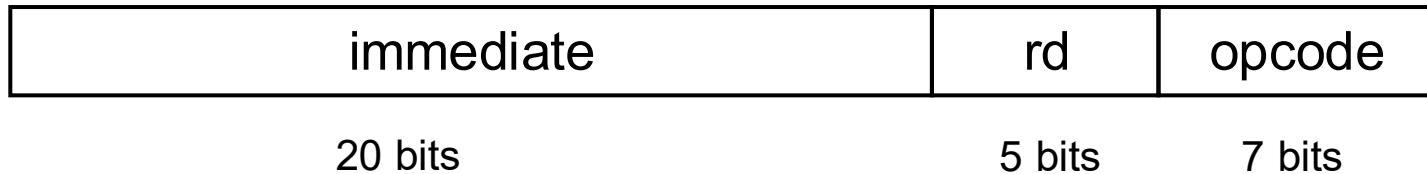
32-Bit Immediate Operands

- Or use **add upper immediate to PC** (`auipc`) to create a PC relative address
 - Forms a 32-bit signed offset by placing the 20-bit immediate in the upper 20 bits and clearing the lower 12 bits
 - Combined with an instruction like `addi`, `auipc` can form a PC-relative address anywhere in the 32-bit address space

`auipc rd, imm`

- Forms a 32-bit offset from the 20-bit immediate, filling in the lowest 12 bits with zeros
- Adds this offset to the PC
- Places the result in register `rd`

U-format Instructions



■ Upper immediate instructions

- `rd`: destination register number
- `immediate`: 20-bit immediate to be placed in upper 20 bits of register `rd`
 - The 32-bit immediate consists of the upper 20 bits encoded in the instruction and 0's in the lower bits
- `opcode`: the operation code

U-format Instructions

- So, in this case, after the instruction executes, register s5 (rd) holds the value 0x8CDEF000

Assembly

Field Values

Machine Code

	imm _{31:12}	rd	op	imm _{31:12}	rd	op	
lui s5, 0x8CDEF	0x8CDEF	21	55	1000 1100 1101 1110 1111	10101	011 0111	(0x8CDEFAB7)
lui x21, 0x8CDEF	20 bits	5 bits	7 bits	20 bits	5 bits	7 bits	

- The use of separate U-format instructions ensures that large constants can be split across multiple instructions
 - Reducing the need for complex encoding schemes and maintaining the regularity of the instruction set

R, I, S and U: What We Have So Far

- **All formats start with an opcode (7 bits)**
 - The funct3/funct7 fields refine the operation within the same opcode
- **R-format has two source registers**
- **I-format uses one source register and a 12-bit immediate**
- **S-format uses two source registers and a 12-bit split immediate**
- **U-format loads large constants (upper 20 bits) to a register**

Takeaway

- A recognition that no single design aspect can be maximized in isolation without considering its impact on other aspects

Design Principle 4: Good design demands good compromises

Different formats complicate decoding, but allow 32-bit instructions uniformly
Keep formats as similar as possible

Control Flow Instructions

- **Branches and JAL use PC-relative addressing**
 - So, their formats prioritize encoding a large signed offset rather than an extra register field
- **B-type and J-type reuse the field layout of S- and U-formats, respectively, while rearranging the immediate bits**
 - They must include register fields and a large immediate, but their **immediate fields are rearranged** across the instruction word
- **Immediate bits are **split and re-ordered** so the instruction can pack a wider PC-relative offset while still fitting $rs1/rs2$ or rd in the same 32-bit word**

Control Flow Instructions

- **The encoded immediate omits the least-significant bit**
 - The resulting offset is expressed in multiples of 2 bytes
 - The implicit zero allows the encoded bits to represent a larger byte offset
- **The hardware implications will become clearer when we study instruction fetch and address generation**
 - We'll cover that in the next lectures

B-format Instructions

- **Revisiting the bne instruction**
 - Causes a branch to a target address if the values in two registers are not equal (if $rs1 \neq rs2$, $PC = PC + offset$)

C Code

```
if (i == j)
    f = g + h;
```

```
else
    f = f - i;
```

RISC-V assembly code

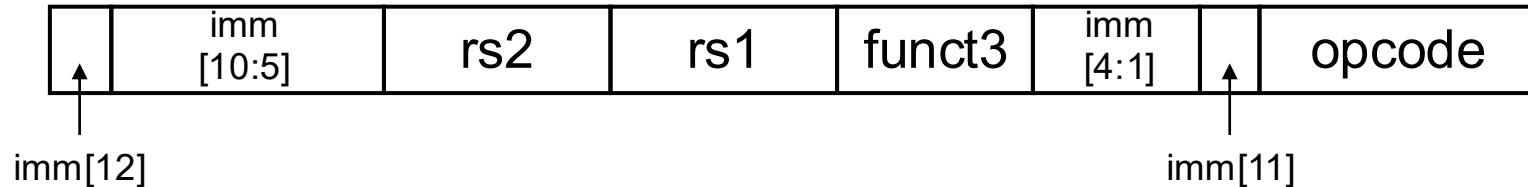
```
# s0 = f, s1 = g, s2 = h
# s3 = i, s4 = j
        bne    s3, s4, L1
        add    s0, s1, s2
        j      done
```

```
L1:
        sub    s0, s0, s3
done:
```

B-format Instructions

- **Most branch targets are a nearby instruction**
 - The branch offset is sign-extended, allowing both forward and backward jumps
- **The 12 encoded immediate bits determine the branch offset (relative to the branch instruction)**
 - The encoded immediate represents a signed branch offset with a range of -4096 to +4094 bytes
- **The branch offset is expressed in multiples of 2 bytes**
 - Bit 0 is implicit and therefore not encoded
 - The encoded bits are rearranged across the instruction

B-format Instructions



- **Branch instructions specify:**
 - `rs1`: first operand register
 - `rs2`: second operand register
 - `imm`: 12 encoded immediate bits
 - Split so that `rs1` and `rs2` fields always in the same place

B-format Instructions

- **Resemble S-format**

- Differ only in immediate encoding
- The branch offset is expressed in multiples of 2 bytes
- Bit 0 is implicit and not encoded

- **This preserves alignment and expands range**

- The sign bit of the immediate value (`imm[12]`) is in the same location of S-format
- As are the middle bits of the immediate value (`imm[10:1]`)
- The encoded immediate provides a **13-bit signed branch offset**, with bit 0 implicitly zero

B-format Instructions

- **Branch targets are always at least 2-byte aligned**
 - The branch offset is therefore always a multiple of 2 bytes
 - **Bit 0 is always zero** and does not need to be encoded
 - The branch target offset is reconstructed with bit 0 = 0
- **Why not omit bit 1 as well?**
 - The RISC-V C extension adds 16-bit compressed instructions
 - When C is implemented, instruction addresses can be aligned to 2 bytes rather than only 4 bytes
 - Therefore, the branch encoding must preserve bit 1 of the target offset

B-format Instructions

- In this example, the branch target address is four instructions after the beq instruction
 - That is, $4 \times 4 = 16$ bytes past beq

#Address	# RISC-V Assembly
0x70	beq s0, t5, L1
0x74	add s1, s2, s3
0x78	sub s5, s6, s7
0x7C	lw t0, 0(s1)
0x80	L1: addi s1, s1, -15

Arrows indicate the sequence of instructions: 1 (beq), 2 (add), 3 (sub), 4 (lw), and 5 (addi).

L1 is 4 instructions (i.e., **16 bytes**) past beq

imm _{12:0} = 16	0	0	0	0	0	0	0	0	1	0	0	0	0
bit number	12	11	10	9	8	7	6	5	4	3	2	1	0

Assembly	Field Values						Machine Code						
beq s0, t5, L1 beq x8, x30, 16	imm _{12,10:5}	rs2	rs1	funct3	imm _{4:1,11}	op	imm _{12,10:5}	rs2	rs1	funct3	imm _{4:1,11}	op	
	0000 000	30	8	0	1000 0	99	0000 000	11110	01000	000	1000 0	110 0011	(0x01E40863)
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	

J-format Instructions

- **Revisiting the jal instruction**
 - Jumps to a target address and stores the return address (current PC + 4) in the specified register ($rd = PC + 4$; $PC = PC + offset$)

C Code

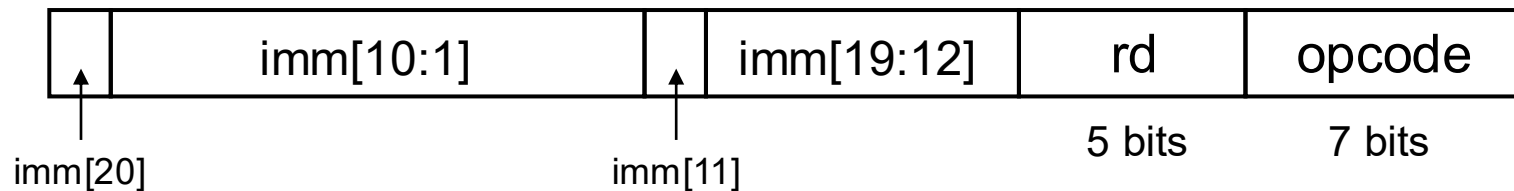
```
int main() {  
    simple();  
    a = b + c;  
}  
  
void simple() {  
    return;  
}
```

RISC-V assembly code

```
0x00000300 main:    jal    ra, simple  
0x00000304          add    s0, s1, s2  
...               ...  
  
0x0000051c simple: jalr    zero, 0(ra)
```

J-format Instructions

- **JAL encodes a 20-bit immediate, producing a 21-bit signed offset with bit 0 implicitly zero**
 - Procedure calls may require targets beyond the ± 1 MiB JAL range



- **The immediate in J-format is scrambled in a similar manner to B-format**
 - Because control-flow targets must be at least 2-byte aligned

J-format Instructions

- J-format instructions jump to an instruction address that is relative to the current PC
 - The least significant bit is not encoded in the instruction because it is always 0

# Address	RISC-V Assembly
0x0000540C	jal ra, func1
0x00005410	add s1, s2, s3
...	...
0x000ABC04	func1: add s4, s5, s8
...	...

func1 is **0xA67F8** bytes past jal

$$0xABC04 - 0x540C = \mathbf{0xA67F8}$$

imm = 0xA67F8

0	1	0	1	0	0	1	1	0	0	1	1	1	1	1	1	1	0	0	0	X
20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

bit number

Assembly

```
jal ra, func1
jal x1, 0xA67F8
```

Field Values

imm _{20,10:1,11,19:12}	rd	op
0111 1111 1000 1010 0110	1	111
20 bits	5 bits	7 bits

Machine Code

imm _{20,10:1,11,19:12}	rd	op	
0111 1111 1000 1010 0110	00001	110 1111	(0x7F8A60EF)
20 bits	5 bits	7 bits	

J-format Instructions

- **For longer jumps, e.g., to a 32-bit absolute address**
 - `lui`: load address[31:12] to temporary register (**U-format**)
 - `jalr`: add address[11:0] and jump to target (**I-format**)

Jump to 0x12345678

```
lui    t0, 0x12345
jalr   ra, 0x678(t0)
```

- **Key idea**
 - `jal` → PC-relative jump
 - `jalr` → register-indirect jump
 - `lui + jalr` → reach any 32-bit address

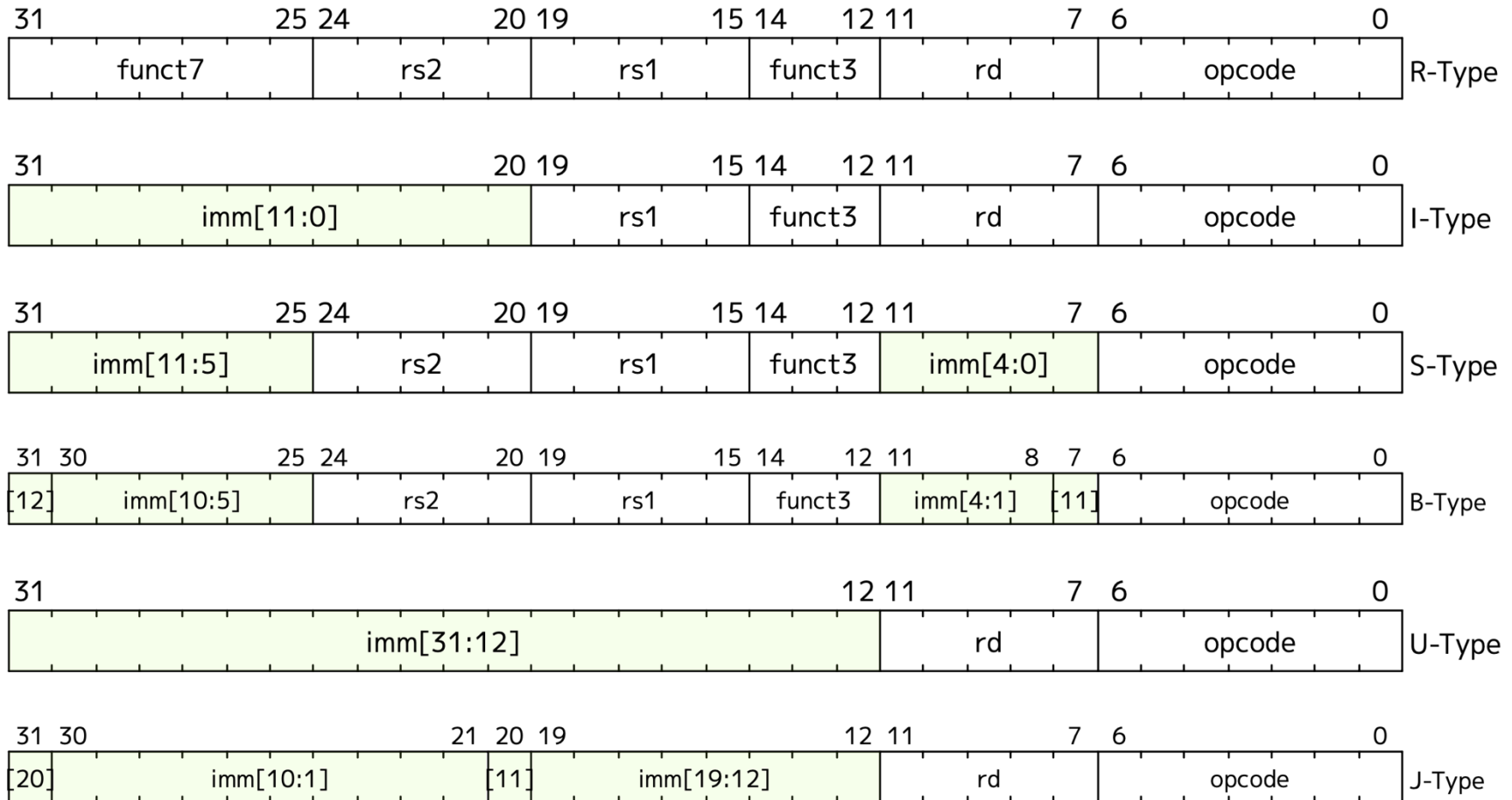
Data Processing vs Control Flow

- **R, I, S, and U-formats handle data processing and addressing**
- **B- and J-formats are specialized for control-flow instructions**
 - Where the next instruction address depends on a relative offset rather than data registers
- **Their unique layouts maximize immediate range while keeping the other fields consistent**

RISC-V Encoding Summary

- All instructions in the base RV32I ISA are **uniformly 32 bits in length**
 - This simplifies the instruction fetch and decoding process
- **Consistent use of fields**
 - **opcode (7 bits)**: Present in all instruction types, always in the same position (least significant 7 bits)
 - **rd (5 bits)**: Destination register field, consistently used in R-type, I-type, U-type, and J-type instructions
 - **rs1 (5 bits)**: First source register field, used in R-type, I-type, S-type, and B-type instructions
 - **rs2 (5 bits)**: Second source register field, used in R-type, S-type, and B-type instructions

RISC-V Encoding Summary



Design Principle: Simplicity Favors Regularity

- **All instruction formats except R-type contain immediate-related fields**
 - I-type: 12-bit immediate
 - S/B-type: split immediate
 - U-type: 20-bit upper immediate
 - J-type: 20-bit jump offset
- **Key observations**
 - Although the size and purpose of immediates differ, their bits are placed as consistently as possible
 - For signed immediates, the sign bit is always located in the most significant instruction bit

Addressing Modes

- An addressing mode defines **how an instruction specifies its operands**
- **RISC-V uses four main modes:**
 - Register — operands are in registers
 - Immediate — an operand is encoded in the instruction
 - Base — memory address = register + immediate offset
 - PC-relative — target address = PC + immediate offset

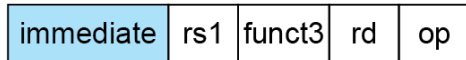
Addressing Modes

- **RV32I keeps addressing modes simple and regular**
 - The first three specify how **data operands are read or written**
 - PC-relative addressing computes a **target address relative to the PC**
 - Complex memory addressing modes, such as indexed addressing, are avoided to simplify hardware and pipelining

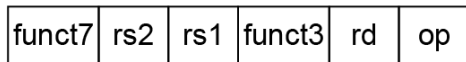
Mode	Description	Example
Register	Operand comes from a register	<code>add x1, x2, x3</code>
Immediate	Constant value encoded in the instruction	<code>addi x1, x2, 10</code>
Base addressing	Memory address = register + signed offset	<code>lw x1, 8(x2)</code>
PC-relative	Address computed from PC + signed offset	<code>beq x1, x2, label</code> <code>jal x1, label</code>

Addressing Modes

1. Immediate addressing



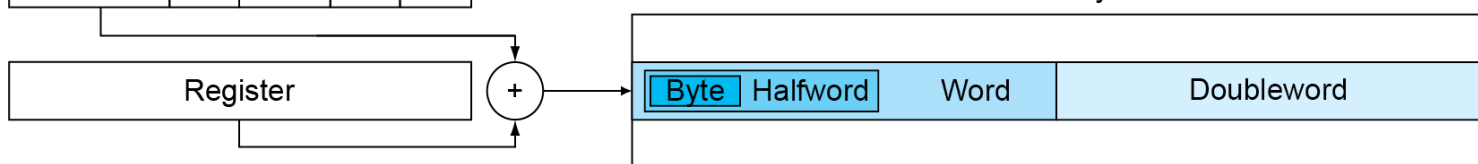
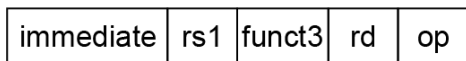
2. Register addressing



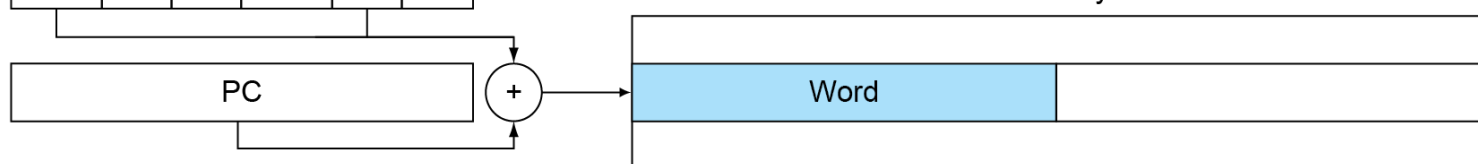
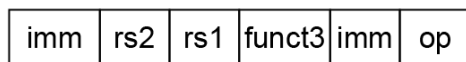
Registers

Register

3. Base addressing



4. PC-relative addressing



Interpreting Machine Language Code

- **A disassembler translates machine code into assembly language**
 - Replacing binary opcodes with human-readable mnemonics
- **Applications**
 - Reverse engineering and malware analysis
 - Program debugging and inspection
 - Verification of compiler-generated code
 - Understanding how software maps to hardware instructions

Interpreting Machine Language Code

- **Disassembly reconstructs instructions, not the original source code**
 - Variable and function names are usually unavailable
 - Data types and comments are lost
 - Compiler optimizations may inline functions, reorder instructions, eliminate variables and temporary values
- **Result**
 - The generated assembly is readable but may differ significantly from the original program

Interpreting Machine Language Code

- To interpret machine language, one must decipher the fields of each 32-bit instruction word
- E.g., translate the following machine language code into assembly language
 - 0x41FE83B3
 - 0xFDA48293
- Different instructions use different formats, but **all formats share a 7-bit opcode field**
 - Thus, the best place to begin is to look at the opcode to determine the instruction's format

Interpreting Machine Language Code

Instruction	opcode	funct3	funct7	Type
add	0110011 (51)	000 (0)	0000000 (0)	R-Type
sub	0110011 (51)	000 (0)	0100000 (32)	R-Type
and	0110011 (51)	111 (7)	0000000 (0)	R-Type
or	0110011 (51)	110 (6)	0000000 (0)	R-Type
addi	0010011 (19)	000 (0)	-	I-Type
beq	1100011 (99)	000 (0)	-	B-Type
bne	1100011 (99)	001 (1)	-	B-Type
lw	0000011 (3)	010 (2)	-	I-Type
sw	0100011 (35)	010 (2)	-	S-Type
jal	1101111 (111)	-	-	J-Type
jalr	1100111 (103)	000 (0)	-	I-Type
lui	0110111 (55)	-	-	U-Type

Interpreting Machine Language Code

- First, we represent each instruction in binary and look at the 7 least significant bits to find the opcode for each instruction
- Then, extra fields (funct3 and funct7) tell operation

0x41FE83B3: 0100 0001 1111 1110 1000 0011 1011 0011

op = 51, funct3 = 0: add or sub (R-type)

funct7 = 0100000: sub

0xFDA48293: 1111 1101 1010 0100 1000 0010 1001 0011

op = 19, funct3 = 0: addi (I-type)

Interpreting Machine Language Code

- By grouping the remaining bits into the R-type and I-type formats, we get the assembly code equivalent to the two machine instructions

	Machine Code						Field Values						Assembly
	funct7	rs2	rs1	funct3	rd	op	funct7	rs2	rs1	funct3	rd	op	
(0x41FE83B3)	0100 000	11111	11101	000	00111	011 0011	32	31	29	0	7	51	sub x7, x29, x31 sub t2, t4, t6
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	
	imm _{11:0}	rs1	funct3	rd	op	imm _{11:0}	rs1	funct3	rd	op			
(0xFDA48293)	1111 1101 1010	01001	000	00101	001 0011	-38	9	0	5	19	addi x5, x9, -38 addi t0, s1, -38		
	12 bits	5 bits	3 bits	5 bits	7 bits	12 bits	5 bits	3 bits	5 bits	7 bits			

RISC-V Design Principles

1. Simplicity favours regularity

- All instructions have a single size
- Three register operands in register-register arithmetic instructions
- Keeping the register fields in the same place in each instruction format

2. Smaller is faster

- Use just 32 registers

RISC-V Design Principles

3. Make the common case faster

- PC-relative addressing for conditional branches and direct jumps
- Immediate addressing for constant operands

4. Good design demands good compromises

- Providing for larger addresses and constants in instructions Vs keeping all instructions the same length

Concluding Remarks

- **RISC-V uses six instruction formats to encode its operations: R, I, S, B, U and J**
 - Each format defines how the 32 instruction bits are divided among the opcode, registers and immediate fields
 - The formats are designed for regularity and efficient decoding
 - Different formats provide the fields needed for arithmetic, memory access and control flow
- **Why it matters**
 - The instruction format determines how the processor interprets the bits of an instruction